Integrated Automated Feeding and Watering System with Centralized Data Monitoring for Commercial Piggeries

An Undergraduate Thesis Proposal
Presented to the School of Engineering
University of San Carlos
Cebu City, Philippines

In Partial Fulfillment of the Requirements for the Degree
Bachelor of Science in Computer Engineering

By

Alvin Joseph S. Macapagal

Jan Dave Q. Campañera

Vaun Michael C. Pace

Paul Andrew F. Parras

Lorenz Anthony L. Soriano

April 2025

Integrated Automated Feeding and Watering System with Centralized Data Monitoring for Commercial Piggeries

Pace, Vaun Michael C.[1], Parras, Paul Andrew F.[1], Soriano, Lorenz Anthony L.[1],
Macapagal, Alvin Joseph S.[2], Campañera, Jan Dave Q.,[2]
[1]*Undergraduate Student, Department of Computer Engineering, University of San Carlos, Talamban, Cebu City, Philippines*
[2]*Faculty Member, Department of Computer Engineering, University of San Carlos, Talamban, Cebu City, Philippines*
pacevaun@gmail.com, parraspaul13874@gmail.com, lorenzal.soriano@gmail.com,
ajsmacapagal@usc.edu.ph, jdcampanera@usc.edu.ph

***Abstract*—Manual feeding and inadequate health monitoring in commercial piggeries contribute to feed waste, undetected illness, and preventable livestock losses. This paper presents the design and development of an integrated, hardwired automated feeding and watering system with centralized data monitoring for commercial piggeries. The system employs a main control module connected to multiple feeder modules via a wired network, integrating weight sensors, water flow sensors, and air quality sensors to ensure precise feed rationing and continuous environmental monitoring. Collected data is processed and displayed through a local user interface, enabling caretakers to track consumption patterns and detect early signs of illness in real time. By eliminating reliance on manual observation and wireless communication, the system offers a reliable and scalable solution for improving biosecurity, animal health management, and operational efficiency in swine production.**

***Index Terms*—Livestock Automation, Precision Feeding, Biosecurity, Real-time Monitoring**

I. Introduction

Livestock farming is a significant component of global agriculture, food security, and economic activity. As the global population surpasses 8.2 billion in 2025, livestock farming continues to play a vital role in food security, rural livelihoods, and international trade [1] [2]. Livestock also has substantial economic importance. The gross value of global livestock production is projected to increase by 14% over the next decade, with the livestock sector leading this expansion with a 16% rise [3]. Global dietary preferences are also shifting toward nutrient-rich animal proteins due to rising incomes and urbanization, especially in middle-income countries in Asia and Latin America [2] [3]. By 2034, daily per capita consumption of fish and livestock products is expected to increase by 25% in lower-middle-income nations [3] [2].

Within the global livestock industry, swine production remains a major source of animal protein and an important contributor to food security [2]. Pork continues to hold a significant share of global meat production, ranking second only to poultry, with an estimated output of 116.27 million metric tons in 2025 [4]. This level of production is largely driven by a few major producers, particularly China, which maintains an inventory of approximately 400 million pigs and accounts for about 49% of total global pork production. The European Union and the United States follow, contributing 19% and 11%, respectively. [4]. These figures show the scale and concentration of the global pork industry, but they also highlight the importance of swine farming beyond large-scale producers. In many developing countries, pig production remains closely tied to local food supply, household income, and rural livelihoods. In the Philippines, for example, the swine industry is valued at around ₱191 billion and accounts for 78% of the country's total livestock output [2] [5] [6]. Around 71.1% of the Philippine hog inventory is managed by backyard smallholder farmers, indicating the role of piggery farming in local economic activity [6].

The global swine industry is susceptible to transboundary diseases and production inefficiencies, making consistent health monitoring necessary for early disease detection, economic sustainability, improved animal welfare, and reduced dependence on antibiotics in agriculture [7]. Although manual inspection of pens, pig behavior, and air quality remains common practice, modern swine farming is steadily shifting toward advanced, continuous monitoring technologies to achieve these goals [7]. The Philippines, where the swine industry is valued at around ₱191 billion and accounts for 78% of the country's total livestock output [6] [2], has faced growing pressure to modernize farm-level health monitoring practices in light of recurring disease outbreaks and sustained livestock losses in recent years.

Manual pig monitoring is time-consuming and prone to error, especially when caregivers are overworked. A pig's health status is closely linked to its feeding and drinking behavior, as measurable deviations in dietary intake are often among the first observable indicators of illness — yet these deviations go undetected without a structured data-recording system. In a case study conducted in a piggery in Tuburan, Cebu, the inability of caretakers to constantly monitor every pig led to the death of 20 nursery pigs in a single month [8].

Furthermore, unmonitored air quality in piggery enclosures poses a risk of respiratory illness among swine, adding to the effects of insufficient caretaker oversight [6] [2]. Meanwhile, commercial operations in regions such as Central Luzon and CALABARZON are increasingly adopting climate-controlled, tunnel-ventilated housing with digital sensors as the sector modernizes, with these monitoring conditions helping reduce pre-weaning mortality from 10% to 9.43% while improving productivity and reproductive performance [6] [2].

II. Statement of the Problem

Inconsistent feeding and drinking practices in pigs can lead to significant health risks and feed wastage [3]. Early detection of illness through physical or manual observation is often unreliable because subtle symptoms may go unnoticed [4]. Without a consistent feed and water tracking system, caretakers must rely on labor-intensive observation, which can lead to inaccurate records and delays in identifying symptoms or other issues. A pig's health status is closely linked to its feeding and drinking behavior, as measurable deviations in dietary intake are often among the first observable indicators of illness [8]. However, without a data-recording system, individual feeding patterns are not established or tracked over time, making it difficult to distinguish normal behavioral variation from significant decline. In addition, unmonitored air quality in piggery environments poses an unnecessary and preventable risk of common respiratory illnesses such as coughs and colds among pigs [2].

This study addresses these issues by proposing a system that establishes per-module consumption baselines using a rolling time window and flags deviations beyond a defined threshold as potential signs of abnormal behavior requiring immediate caretaker attention. Air-quality readings from integrated sensors are also compared against safe-range standards, with alerts generated when toxicity levels within the pig-pen environment exceed acceptable limits. This approach aims to reduce reliance on subjective visual observation and provide a more consistent and reliable basis for early health monitoring.

III. Goals and Objectives

The main goal of this project is to design and develop an integrated automated swine feeding and watering system with centralized data monitoring and control.

To achieve this goal, the system must fulfill the following specific objectives:

- Implement a programmable central control unit responsible for managing the main feed storage, executing scheduled feed dispensing, monitoring water levels, and triggering water dispensing when the level falls below the required threshold.
- Design individual feeder modules capable of monitoring feed consumption, water level, and ammonia levels.
- Develop a user interface on the central unit for real-time system monitoring, historical data visualization, alert display, and feeding schedule configuration.
- Integrate a multi-channel alert system that notifies caretakers of consumption or environmental anomalies through both a local touchscreen dashboard and remote SMS notifications delivered via a LoRa-GSM gateway.

IV. Scope and Limitations

This study focuses on the design and development of an automated feeding and watering system with data monitoring for commercial piggeries. The system is intended to support swine management by combining scheduled feed and water dispensing, sensor-based environmental monitoring, and digital record-keeping within a farrowing piggery prototype setup.

This study is limited to the following:

- The system does not perform direct veterinary diagnosis or laboratory disease confirmation.
- Implementation is limited to the system configuration used, with dimensions based on the farrowing pen setup; performance may vary across different farm layouts.
- System operation may be affected by interruptions in electrical power.
- Air quality monitoring is limited to sensors integrated within the feeder modules and covers only the immediate environment of each pen, not the entire piggery facility.
- Remote SMS alerts are limited by GSM signal availability, SIM card functionality, and mobile network service. During cellular service interruptions, alerts remain available through the local dashboard.

V. Significance of the Study

This study holds significant value for multiple sectors, ranging from small-scale backyard farmers to major commercial piggery operators, as well as broader agricultural and technological stakeholders.

Swine Farmers and Caretakers

By replacing manual feeding with a sensor-based dispensing and monitoring system, the proposed solution directly addresses feeding problems such as inconsistent rationing, excess feed distribution, feed wastage, and difficulty tracking actual feed intake per pen. The system measures the amount of feed dispensed and consumed through the load cell, allowing caretakers to compare current consumption with previous feeding records. When feed intake drops below the established baseline, the system flags the change as a feeding irregularity that requires inspection. In this way, the system helps improve feeding consistency, reduce feed waste, and provide caretakers with a clearer basis for identifying pens that need attention.

Animal Health and Biosecurity

The proposed system provides caretakers with continuous records of feed consumption, water level, and air-quality conditions in each monitored pen. These records allow caretakers to compare current pen conditions with previous readings instead of relying only on manual observation. When the system detects abnormal feed consumption, low water level,

or air-quality readings beyond the set threshold, it generates an alert for caretaker inspection.

Through automated alerts and structured data records, caretakers can respond more promptly to feeding irregularities, water supply issues, or unsafe air-quality conditions. These alerts provide caretakers with timestamped information on the type and location of the detected abnormal condition, allowing them to inspect the affected pen based on recorded sensor readings rather than manual observation alone. In this way, the system supports organized record-keeping and consistent monitoring of feeding, watering, and environmental conditions.

The Philippine Swine Industry

With approximately 71.1% of the Philippine hog inventory managed by smallholder backyard farmers, the local swine sector remains highly vulnerable to disruptions caused by disease and inefficient farm management. A reliable, centralized automated system could help stabilize domestic pork production, reduce dependence on imported meat, which reached a record 1.64 million metric tons in 2025, and support the national goal of rebuilding the country's swine population to pre-ASF levels.

Field of Agricultural Automation and Embedded Systems

This study contributes to the growing body of research on IoT and precision agriculture applied to livestock management. By demonstrating a fully hardwired, sensor-integrated feeding and monitoring system designed specifically for the Philippine piggery context, the project provides a practical reference for future engineering work in agricultural automation, particularly in environments where wireless reliability may be a concern.

Future Researchers

The system's data logging and analytics capabilities generate structured, timestamped records of feed consumption, water intake, and environmental conditions. This dataset can serve as a foundation for future studies on swine behavioral patterns, machine learning-based health prediction models, and the broader integration of precision livestock farming technologies in developing agricultural economies.

VI. Review of Related Literature

A. Automated Feeding and Watering Systems

Automated feeding and watering systems in commercial piggeries are designed to deliver controlled and consistent amounts of feed and water using sensor-based monitoring and programmable control logic. These systems aim to reduce feed wastage and improve efficiency by replacing manual, labor-dependent practices with precise and repeatable operations [7], [9].

Literature emphasizes that inconsistencies in feeding and drinking behavior can lead to health deterioration and inefficient resource utilization [2], [3]. As a result, automated systems are designed to standardize feeding schedules and quantities while ensuring that consumption is monitored continuously. A key consideration in system design is maintaining consistency across multiple feeding points, which requires coordinated control and reliable data exchange within the system.

B. Health Monitoring Through Behavioral and Consumption Data

Behavioral monitoring through feeding and drinking patterns is widely recognized as a non-invasive method for early disease detection in swine. Changes in consumption behavior often occur before visible clinical symptoms, making them valuable indicators of potential health issues [8], [10].

Studies such as Dingcong et al. demonstrate that tracking feeder and drinker interactions can provide insight into animal health and behavior [8]. More broadly, research highlights the importance of establishing baseline consumption patterns and identifying deviations over time [10], [11]. These deviations can be analyzed using time-series approaches, where historical data is used to define normal behavior and detect anomalies.

Without structured data collection and analysis, caretakers rely on subjective observation, which may result in delayed detection of illness. The literature supports the use of continuous monitoring systems to provide objective and data-driven assessment of animal health conditions [8], [11].

C. Air Quality Sensing in Swine Environments

Environmental conditions within piggery enclosures play a significant role in animal health and productivity. Poor air quality has been associated with respiratory illnesses and reduced overall welfare in swine populations [2], [12].

Research indicates that monitoring environmental parameters enables early identification of unsafe conditions and supports timely intervention. Air quality data also complements behavioral monitoring, as environmental stressors can influence feeding and drinking patterns [10], [12]. This relationship highlights the importance of integrating environmental and behavioral data to achieve a more comprehensive understanding of animal health.

Existing studies also explore advanced approaches such as sound-based detection of respiratory symptoms, further reinforcing the role of non-invasive monitoring techniques in livestock management [12].

D. Centralized Data Monitoring and User Interface Design

Centralized monitoring systems are widely used in agricultural automation to aggregate data from multiple sensing points and provide a unified view of system status. These systems enable real-time monitoring, data logging, and alert generation, supporting informed decision-making by farm operators [7], [11].

The literature emphasizes the importance of accessible and intuitive user interfaces that present complex data in a simplified manner. Features such as real-time dashboards, historical trend visualization, and threshold-based alerts are commonly identified as essential components of effective monitoring systems [10].

In addition, centralized systems reduce reliance on manual observation and improve the accuracy and consistency of recorded data. Continuous monitoring technologies have been shown to enhance productivity and reduce labor demands in modern swine farming operations [2], [6].

E. Disease Outbreaks and Farm-Level Monitoring in Swine Production

Disease outbreaks such as African Swine Fever have emphasized the importance of biosecurity and consistent farm-level monitoring in swine production. However, automated monitoring systems do not replace veterinary diagnosis or laboratory confirmation. Their role is to support daily farm management by continuously recording feeding behavior, water availability, and environmental conditions that may indicate abnormal pen conditions requiring caretaker inspection. In this context, the proposed system contributes to biosecurity-oriented management by improving the consistency of monitoring and record-keeping, rather than by directly detecting or preventing ASF. [13].

F. Scalability and Deployment Considerations in Philippine Piggeries

The structure of the Philippine swine industry, characterized by a combination of commercial farms and smallholder operations, presents unique challenges for technology adoption. With a large proportion of hog production managed by backyard farmers, systems must be adaptable, scalable, and accessible [6].

Literature highlights the need for flexible system designs that can accommodate varying farm sizes and operational conditions. Continuous monitoring technologies are increasingly being adopted in modern farming environments, demonstrating their potential to improve productivity and animal welfare [2], [6].

Additionally, non-invasive monitoring approaches, including sound-based and image-based techniques, provide scalable methods for observing animal health across large populations [8], [14]. These approaches expand the range of available tools for precision livestock farming and support the gradual modernization of the sector.

VII. Methodology

This chapter presents the conceptual framework underpinning the system's design philosophy, followed by a detailed account of the system architecture, hardware construction, feed hopper and trough volume calculations, feed and water dispensing mechanism, environmental and consumption sensing strategy, wired communication protocol, and system performance evaluation metrics.

A. Conceptual Framework

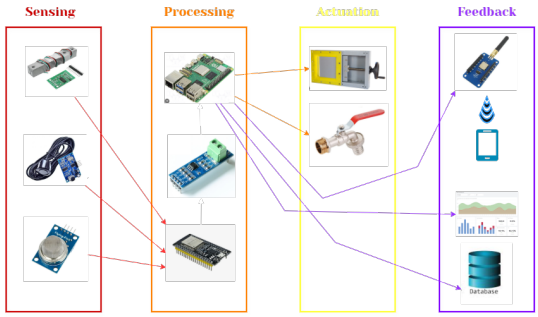

Fig. 1. Conceptual Framework

As illustrated in Figure 1, the system is conceptualized as a closed-loop precision livestock management platform grounded in two theoretical underpinnings: Precision Agriculture Theory—which frames sensor-driven, individualized resource delivery as the basis for efficient swine production—and Feedback Control Systems Theory, which models each feeder module as a node whose output (feed and water dispensed) is continuously logged and compared against defined consumption baselines to flag deviations indicative of illness.

The system's operational logic spans four functional layers:

- **Layer 1 – Sensing (Input):** Each feeder module integrates a load cell with an HX711 24-bit ADC for gravimetric feed measurement, a waterproof ultrasonic sensor (e.g., JSN-SR04T) for water level monitoring, and an MQ-135 electrochemical sensor for ammonia air-quality monitoring.
- **Layer 2 – Processing and Decision (Control):** Each feeder module's ESP32 microcontroller evaluates sensor readings against pre-established rolling-window consumption baselines. Deviations beyond a defined threshold are flagged as health alerts and relayed to the main control module via the RS-485 wired bus. The Raspberry Pi central controller processes incoming data, executes alert logic, and manages the touchscreen dashboard.
- **Layer 3 – Actuation (Output):** Upon a scheduled or manually triggered feeding event, the main control module commands the solenoid-actuated sliding plate valve at the base of the truncated-pyramid hopper to open for a calibrated duration, dispensing a gravimetrically verified quantity of feed into each trough. Water is dispensed through a solenoid valve when the sensor detects the water level has dropped below a predefined threshold. This valve can also be manually overridden using an active-high push button on pin RA0 to ensure rapid caretaker intervention if needed.
- **Layer 4 – Feedback, Logging, and Remote Notification:** All sensor readings, dispensing events, and alerts

are timestamped and stored in a local SQLite database on the Raspberry Pi. In addition to displaying alerts on the local touchscreen dashboard, the system utilizes a LoRa (Long Range) transceiver to transmit critical health and environmental alerts to a remote LoRa-GSM gateway positioned outside the piggery. Upon receiving the LoRa payload, the gateway forwards the alert as an SMS message to the caretaker's mobile phone, ensuring real-time remote notification regardless of the caretaker's location on the farm.

B. System Architecture: Distributed Software Workflow

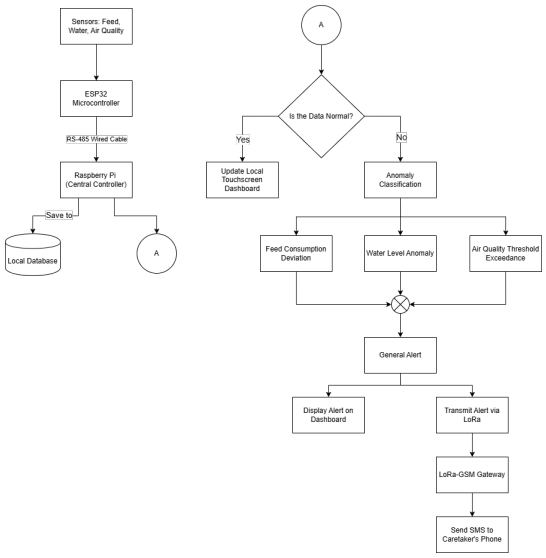

Fig. 2. System Flowchart

The system employs a distributed, hierarchical software architecture to decouple real-time hardware interfacing from high-level data orchestration. As illustrated in Figure 2, the software workflow operates in a continuous loop. Data acquisition begins at the edge nodes where the ESP32 samples the sensors via hardware interrupts. The Raspberry Pi systematically polls these nodes over the RS-485 bus. Upon receiving the telemetry, the Python backend writes the raw data to the SQLite database and passes it to the Analytical Engine. If the data falls within normal baseline parameters, it is pushed to the Flask frontend for live dashboard visualization. If an anomaly is detected, the system simultaneously triggers a visual alert on the local touchscreen dashboard and transmits an alert payload via LoRa to the remote LoRa-GSM gateway, which forwards it as an SMS to the caretaker's mobile phone.

- **Edge Intelligence (Feeder Nodes):** Each node is governed by an ESP32 dual-core microcontroller. One core is dedicated to continuous sensor polling (load cell and flow meter) using hardware interrupts and the HX711 24-bit ADC, while the second core manages a MODBUS RTU slave stack. Telemetry is stored in a local register map, allowing the master to retrieve data asynchronously without blocking sensor acquisition.
- **Central Orchestration (Main Module):** A Raspberry Pi serves as the MODBUS RTU master, implementing a sequential polling loop via the RS-485 bus. The main module is co-located with the central hopper—a 0.18 m^3-capacity truncated-pyramid storage unit—from which feed is gravity-fed through an auger conveyor to individual feeder nodes. The master logic is divided into four primary services:
  - **Poller Service:** Executes MODBUS read commands at 1-second intervals to aggregate per-pen telemetry.
  - **Analytical Engine:** Implements the rolling-window anomaly detection logic, comparing incoming data against the SQLite-persisted baseline.
  - **Command Service:** Dispatches metered dispensing instructions based on the pre-configured feeding schedule or manual overrides.
  - **LoRa Alert Service:** Interfaces with a connected LoRa transceiver module via SPI or UART. When the Analytical Engine flags an anomaly based on the rolling-window baseline, this service encodes the alert data into a lightweight payload and transmits it over the LoRa link to the remote LoRa-GSM gateway for SMS forwarding to the caretaker's mobile phone.
- **Remote LoRa-GSM Gateway:** A secondary edge device positioned outside the piggery in a location with adequate cellular coverage. It receives incoming LoRa alert payloads and forwards them as SMS messages to the caretaker's registered mobile phone number via an onboard GSM module, enabling immediate caretaker notification without requiring physical presence at the main control module.

C. Software Architecture and Technology Stack

The system's software architecture is distributed across the edge feeder nodes and the central control module. The technology stack was deliberately selected to prioritize fault tolerance, low-latency execution, and complete offline capability, ensuring the system remains fully operational without external internet dependencies. The end-to-end data flow pipeline, from sensor acquisition to dashboard visualization, is depicted in Figure 3.

The end-to-end data flow pipeline is illustrated in Figure 3, tracing the path of a single sensor reading from acquisition at the ESP32 edge node through ADC conversion, RS-485 transmission, SQLite persistence, analytical processing, and final visualization on the touchscreen dashboard.

1. Edge Node Firmware (ESP32)

- **Software/Language:** C/C++ utilizing the FreeRTOS (Real-Time Operating System) environment.
- **Why it is used:** C/C++ provides the low-level memory management and fast execution speeds required for high-frequency sensor sampling. FreeRTOS allows the ESP32 to effectively utilize its dual-core processor by dividing tasks into concurrent threads.
- **How it is implemented:** As shown in Figure 4, Core 0 is dedicated strictly to hardware interrupts—specifically

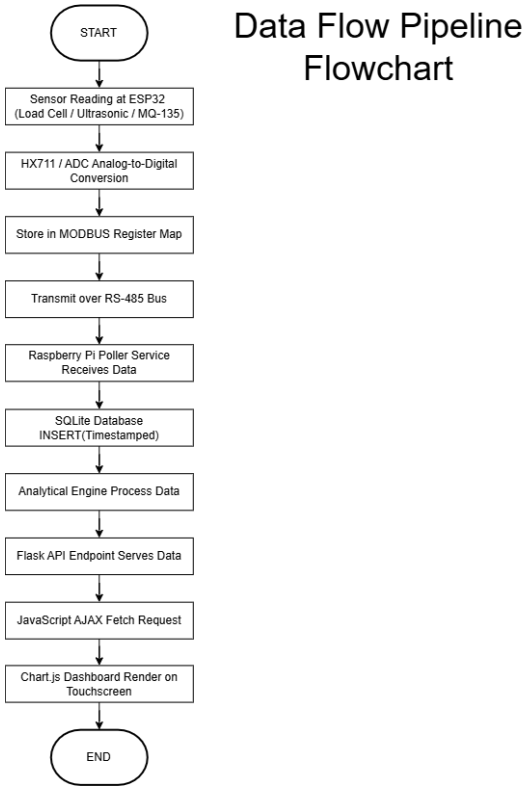

Fig. 3. Data Flow Pipeline Flowchart

counting pulses from the YF-S201 flow meter and reading the HX711 ADC—ensuring no data is lost during execution delays. Core 1 manages the MODBUS RTU slave stack, listening for polling requests from the Raspberry Pi and transmitting the localized sensor data stored in its memory registers.

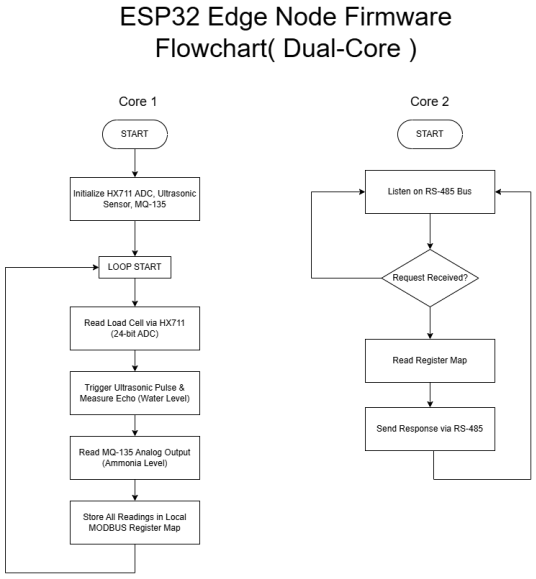

Fig. 4. ESP32 Edge Node Firmware Flowchart (Dual-Core)

The dual-core firmware architecture is depicted in Figure 4. Core 0 continuously cycles through sensor acquisition (load cell, ultrasonic, and MQ-135 readings), while Core 1 concurrently handles MODBUS RTU communication on the RS-485 bus.

2. Central Controller Backend (Raspberry Pi)

- **Software/Language:** Python 3, Flask Web Framework, and PyModbus.
- **Why it is used:** Python offers extensive library support for serial communication and data analysis. Flask is a lightweight WSGI web application framework that does not require heavy dependencies, making it ideal for an embedded Linux environment. PyModbus provides a highly stable, synchronous protocol for RS-485 communication.
- **How it is implemented:** Python acts as the central orchestrator. A background script runs continuous PyModbus loops to query each ESP32 node. The retrieved data is passed through the Python-based Analytical Engine to calculate rolling averages and detect anomalies. Simultaneously, Flask serves the backend API, exposing this data to the local network so the frontend can retrieve it dynamically without requiring a page refresh.

3. Data Persistence (Database)

- **Software:** SQLite.
- **Why it is used:** Unlike heavy relational databases (e.g., MySQL or PostgreSQL) that require dedicated server processes, SQLite is a C-language library that implements a small, fast, self-contained, high-reliability SQL database engine. It writes directly to ordinary disk files, minimizing CPU and RAM overhead on the Raspberry Pi.
- **How it is implemented:** The Python backend executes standard SQL commands to log timestamped telemetry data, system alerts, and feeding schedules. To prevent SD card degradation from excessive write cycles, database commits are batched and executed at defined intervals rather than instantaneously per sensor read.

4. User Interface and Visualization (Frontend)

- **Software/Language:** HTML5, CSS3, vanilla JavaScript, and Chart.js.
- **Why it is used:** This combination ensures a responsive, highly intuitive dashboard. Chart.js is specifically utilized because it renders lightweight, HTML5 canvas-based time-series graphs that do not lag on embedded processors.
- **How it is implemented:** The Raspberry Pi boots directly into a Chromium browser in ”Kiosk Mode” (fullscreen, no navigation bars), pointing to the local Flask server address. JavaScript asynchronous requests (AJAX) continuously pull the latest SQLite data via Flask API endpoints, dynamically updating the Chart.js line graphs and numeric alert gauges so caretakers see real-time shifts in consumption and air quality.

D. Volume and Structural Calculations

The physical dimensions of the central hopper and feeder trough were derived from pig-feed bulk density and a total required storage capacity of 90 kg, using a bulk density of 550 kg/m^3. The minimum required storage volume is:

$$V = \frac{m}{\rho} = \frac{90 \text{ kg}}{550 \text{ kg/m}^3} \approx 0.1636 \text{ m}^3 \tag{1}$$

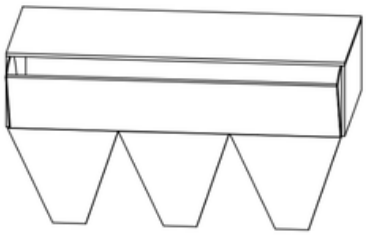

Fig. 5. Main Control Module

An allowance of 0.180 m^3 was adopted as the design target. The overall assembly of the main control module is shown in Figure 5. The central storage hopper is designed as a truncated rectangular pyramid (frustum) with a top face of $L = 1.0$ m $\times$ $W = 0.5$ m and a bottom opening of $a = b = 0.1$ m, consistent with the sliding-plate valve aperture. The slant height of the frustum walls is calculated from:

$$h = \frac{(0.5 \text{ m} - 0.1 \text{ m})}{2} \times \tan(60^\circ) \approx 0.35 \text{ m} \tag{2}$$

The volume of each conical frustum section is given by the prismatoid formula, with corresponding dimensions illustrated in Figure 6:

$$V_{cone} = \frac{h}{3}(A_B + a_b + \sqrt{A_B \cdot a_b}) \tag{3}$$

where $A_B = 0.5$ m $\times$ 0.333 m $= 0.165$ m^2 is the top-face area per section, and $a_b = 0.1$ m $\times$ 0.1 m $= 0.01$ m^2 is the bottom-aperture area. Substituting into (3):

$$V_{cone} \approx 0.02516 \text{ m}^3 \times 3 \approx 0.075 \text{ m}^3 \tag{4}$$

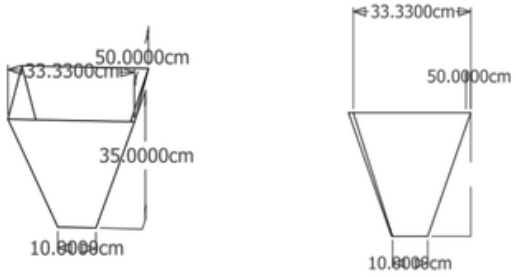

Fig. 6. Cone Dimensions

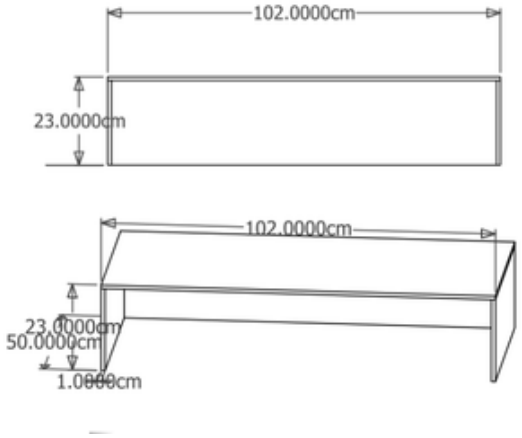

Fig. 7. Box Dimensions

The remaining volume is provided by a rectangular upper reservoir section of height h_{rec}, with the corresponding box dimensions shown in Figure 7. This is computed from:

$$V_{rec} = V_{total} - V_{cones} = 0.180 - 0.075 = 0.105 \text{ m}^3 \tag{5}$$

Solving for the rectangular section height with $L = 1.0$ m and $W = 0.5$ m:

$$h_{rec} = \frac{0.105 \text{ m}^3}{1.0 \text{ m} \times 0.5 \text{ m}} = 0.21 \text{ m} \tag{6}$$

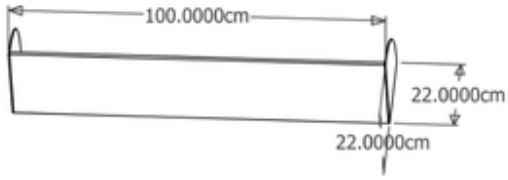

Fig. 8. Hopper Lid Dimensions

The total hopper height is therefore $h_{cone} + h_{rec} = 0.35 + 0.21 = 0.56$ m, housed within an overall footprint of 102 cm $\times$ 50 cm at the top rim, as shown in Figure 8. A separate funnel outlet is provided for each feeder node, directing feed into the trough.

E. Dispensing Mechanism

Feed release from the central hopper is controlled by a solenoid-actuated sliding plate valve (guillotine valve) positioned at the 0.1 m $\times$ 0.1 m bottom aperture of the truncated pyramid, with the complete dispensing process illustrated in Figure 10. A 12 V DC push-pull solenoid drives the plate linearly, opening or closing the aperture under microcontroller command. This valve type was selected for its simplicity, low power consumption, and compatibility with granular feed materials that would jam rotary or butterfly valves.

An auger screw conveyor driven by a NEMA 17 stepper motor (shown in Figure 9) supplements the sliding valve for metered dispensing. By counting motor steps, the ESP32 can

dispense feed in calibrated increments independently of gravity head pressure, improving dose accuracy across varying hopper fill levels. The hinged hopper lid is actuated by a secondary servo motor for automated refill access.

Fig. 9. NEMA 17 Stepper Motor

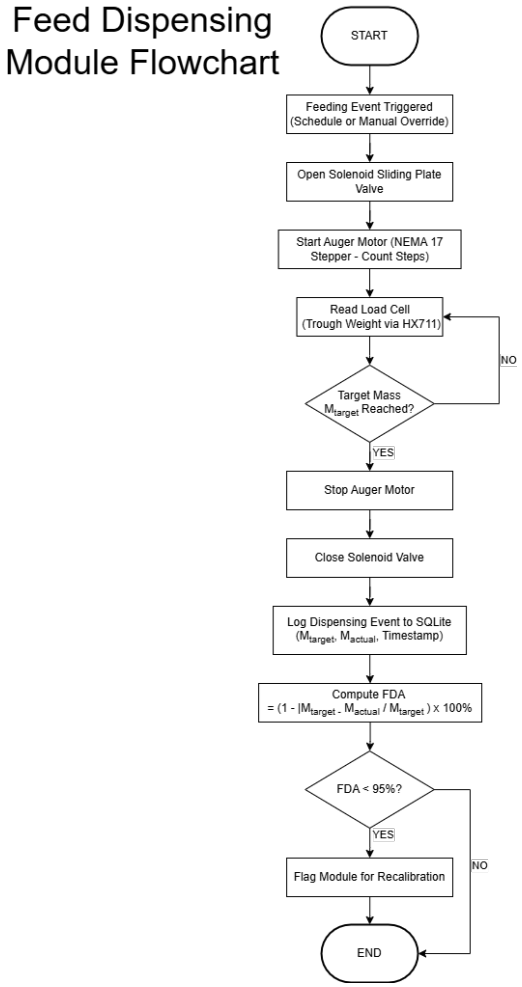

Fig. 10. Feed Dispensing Module Flowchart

The feed dispensing process is summarized in Figure 10. When a feeding event is triggered, the system opens the hopper valve, runs the auger motor, and continuously reads the load cell until the target mass is reached, at which point the motor is stopped, the valve is closed, and the event is logged.

F. Sensing Strategy and Component Selection

Sensor selection was guided by the operating environment of a commercial piggery: high ambient humidity, a corrosive ammonia atmosphere, mechanical vibration from pig activity, and the need for low-cost scalability across multiple feeder nodes.

- **Feed Weight (Load Cell + HX711):** As shown in Figure 11, a strain-gauge load cell rated for the expected trough load is interfaced to an HX711 24-bit sigma-delta ADC. The HX711 communicates with the ESP32 over a dedicated two-wire serial interface, providing 10 or 80 samples per second. The trough weight is logged before and after each feeding event; the delta constitutes the dispensed mass. Consumption is derived by tracking weight loss between feeding cycles.

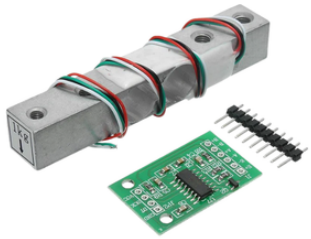

Fig. 11. HX711 & Straight Bar Load Cell

Fig. 12. JSN-SR04T Waterproof Ultrasonic Sensor

- **Water Availability (Level Sensor):** As depicted in Figure 12, a waterproof ultrasonic sensor is mounted above each pen's water trough. The ESP32 triggers an ultrasonic pulse and measures the echo time-of-flight to calculate the distance to the water surface. This determines the current water volume and triggers the solenoid valve to refill the trough when levels fall below the required baseline, ensuring constant water availability. The complete water refill control loop is illustrated in Figure 13.
  The water refill process is illustrated in Figure 13. The system continuously measures the water level; when it

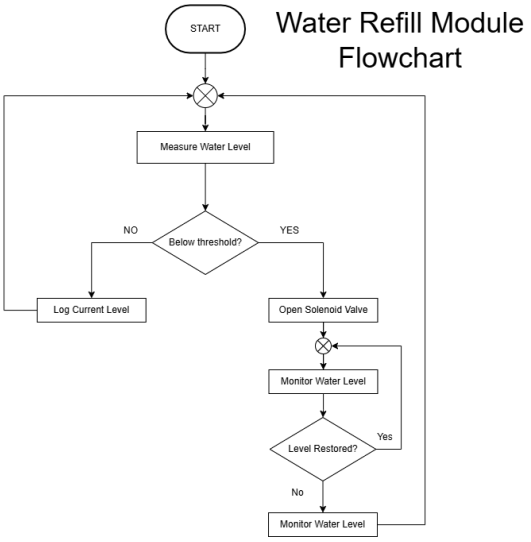

Fig. 13. Water Refill Module Flowchart

drops below the threshold, the solenoid valve opens and the level is monitored until it is restored, at which point the valve closes and the event is logged.

Fig. 14. MQ-135 Air Quality Sensor

- **Air Quality (MQ-135):** The MQ-135 electrochemical sensor, shown in Figure 14, detects ammonia (NH_3), CO_2, and benzene vapors—the primary respiratory hazards in piggery environments arising from urine decomposition. The sensor's analog output is read by the ESP32's onboard ADC and compared against safe-range thresholds; exceedances trigger caretaker alerts on the dashboard.

G. Wired Communication: RS-485 / MODBUS RTU

The system specifies a hardwired network topology to eliminate dependence on wireless signal integrity in metal-framed piggery structures. RS-485 with the MODBUS RTU protocol was selected as the communication backbone for the following reasons:

- Differential signaling on a twisted-pair bus provides high noise immunity in electrically harsh farm environments.
- The standard supports up to 32 nodes on a single bus segment at distances of up to 1,200 m, enabling future expansion of feeder modules without architectural redesign.
- MODBUS RTU is an industry-standard protocol with extensive ESP32 and Raspberry Pi library support, reducing firmware development overhead.

Each feeder-node ESP32 is interfaced to the bus via a MAX485 half-duplex transceiver IC. The Raspberry Pi master polls each slave node at configurable intervals, retrieves sensor data packets, and writes dispensing commands, as illustrated in Figure 15. A unique MODBUS device address is assigned to each feeder module during commissioning.

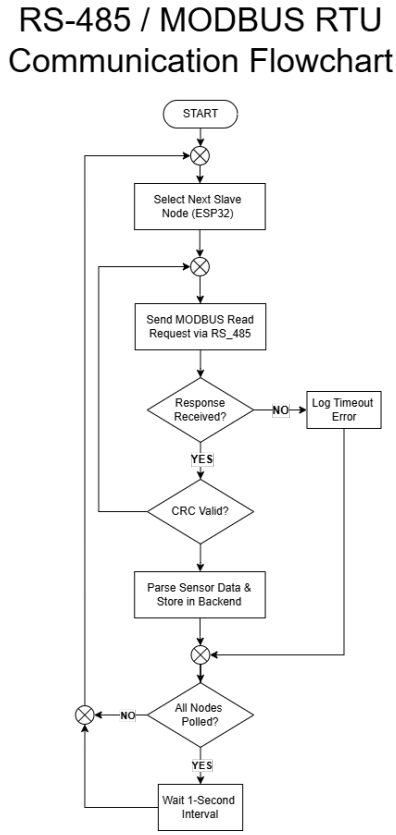

Fig. 15. RS-485 / MODBUS RTU Communication Flowchart

The polling protocol is depicted in Figure 15. For each cycle, the master selects the next slave node, transmits a MODBUS read request, validates the response CRC, and parses the sensor data before moving to the next node. Timeout and retry logic ensures robustness against transient communication failures.

H. Application Layer and Data Persistence

The centralized user interface (UI) is hosted on the Raspberry Pi and rendered through a 7-inch DSI touchscreen mounted on the main control module enclosure. The software stack utilizes a lightweight web-based architecture to ensure low-latency updates:

- **Backend Stack:** A Flask-based web server manages the API endpoints for the dashboard. It interfaces with

an SQLite database for ACID-compliant persistence of timestamped consumption logs, feeding schedules, and air-quality telemetry.

- **Frontend Stack:** A Chromium kiosk browser displays a dashboard built with responsive numeric gauges and time-series charts for real-time monitoring and historical trend visualization. It includes configuration panels for feeding schedules and alert logs for threshold exceedances.
- **Fault Tolerance:** All data is stored locally, ensuring continuous operation during internet outages, which is a primary concern identified in the study's scope and limitations. To address potential grid instability, a UPS module with 18650 lithium cells provides bridge power, ensuring that the SQLite database maintains integrity during short-duration power interruptions.
- **LoRa-SMS Alert Gateway:** A background Python thread runs alongside the Flask backend, with the end-to-end alert pipeline depicted in Figure 16. It continuously monitors the SQLite database for newly generated threshold exceedances. To optimize radio duty cycles and prevent packet collisions, the gateway implements a message-queuing protocol, bundling non-critical alerts and transmitting them via the LoRa link to the remote LoRa-GSM gateway at configured intervals or triggering immediate LoRa transmission for critical air-quality hazards. The remote gateway then forwards each alert as an SMS to the caretaker's mobile phone.

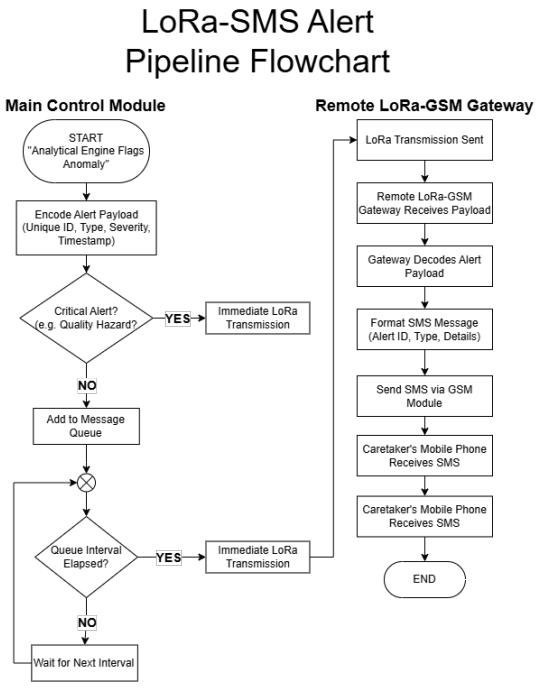

Fig. 16. LoRa-SMS Alert Pipeline Flowchart

The end-to-end alert notification pipeline is illustrated in Figure 16. When the Analytical Engine flags an anomaly, the alert payload is encoded and transmitted via LoRa to the remote gateway, which decodes it and forwards it as an SMS to the caretaker's mobile phone.

I. Consumption Baseline and Anomaly Detection

A pig's health status is closely linked to its feeding and drinking behavior; measurable deviations in dietary intake are often among the first observable indicators of illness [8], [10]. The system establishes per-module baselines using a rolling time-window approach consistent with data-driven livestock monitoring methods reported in the literature [10], [11]. Let C_i denote the feed consumed in session i. The rolling mean over a window of N sessions is:

$$\mu_N = \frac{1}{N} \sum_{i=t-N+1}^{t} C_i \tag{7}$$

A deviation alert is generated when the current session's consumption C_t falls outside the band $[\mu_N - k\sigma, \mu_N + k\sigma]$, where σ is the rolling standard deviation and k is a configurable sensitivity coefficient (default $k = 2$). This statistical approach reduces false positives arising from natural day-to-day variation while reliably flagging behaviorally significant declines [10], [11].

In practice, this algorithm is executed by the Analytical Engine running on the Raspberry Pi, as illustrated in Figure 17. After each feeding session, the Python backend queries the SQLite database to retrieve the most recent $N = 14$ consumption records for the corresponding feeder module. The rolling mean μ_N and standard deviation σ are then computed in memory. If the current session's consumption C_t falls below $\mu_N - k\sigma$, the system classifies the event as a *feed consumption deviation*; if C_t exceeds $\mu_N + k\sigma$, it is flagged as an *abnormal overconsumption event*. In parallel, the same rolling-window logic is applied to the water consumption data derived from ultrasonic sensor readings, and the MQ-135 air-quality readings are compared against a fixed threshold (e.g., ammonia concentration > 25 ppm). When any anomaly is detected, the Analytical Engine simultaneously pushes a visual alert to the Flask-served touchscreen dashboard and dispatches a LoRa payload to the remote LoRa-GSM gateway for SMS delivery to the caretaker. All anomaly events, including their type, severity, timestamp, and the baseline values that triggered them, are persisted in the SQLite database for historical trend analysis.

The anomaly detection logic is summarized in Figure 17. After each feeding session, the system queries the last 14 sessions, computes the rolling mean and standard deviation, and determines whether the current consumption falls within the normal range. If an anomaly is flagged, alerts are simultaneously pushed to the dashboard and transmitted via the LoRa-SMS pipeline.

J. Evaluation Metrics and Verification Protocol

System performance will be assessed against quantitative metrics aligned with the project's specific objectives. These metrics are intended to reflect the reliability of dispensing, sensing, communication, and health-monitoring functions commonly emphasized in precision livestock and embedded

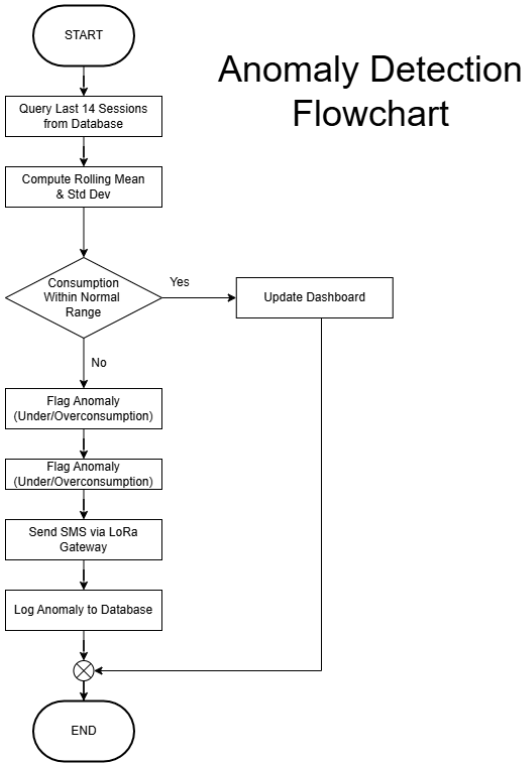

Fig. 17. Anomaly Detection Flowchart

monitoring studies [7], [9], [10]. To ensure system reliability and accuracy prior to full-scale deployment, a rigorous testing protocol will be executed to systematically validate these metrics.

Quantitative Metrics:

- **Feed Dispensing Accuracy (FDA):** Percentage agreement between the target dispensed mass and the gravimetrically measured actual dispensed mass across a calibration trial of 30 dispensing events per module. A minimum passing threshold of $\geq 95\%$ is required for the system to be considered operationally accurate [8], [9]:

$$FDA = \left(1 - \frac{|M_{target} - M_{actual}|}{M_{target}}\right) \times 100\% \quad (8)$$

  In the system, M_{target} is the pre-configured feed mass (in grams) specified in the feeding schedule stored in the SQLite database, while M_{actual} is the gravimetrically measured mass recorded by the load cell after each dispensing event. For each of the 30 calibration trials, the Python backend logs both values and computes the per-event FDA. The overall FDA for a module is reported as the mean across all trials. If the mean FDA falls below 95%, the system flags the module for recalibration of the auger stepper motor step-to-mass conversion factor.
- **Alert Response Accuracy (ARA):** Proportion of simulated consumption anomaly events correctly detected and flagged by the baseline algorithm within one polling cycle. A minimum passing threshold of $\geq 90\%$ is required to validate the reliability of the anomaly detection logic [10], [11]:

$$ARA = \frac{\text{True Positive Alerts}}{\text{Total Induced Events}} \times 100\% \quad (9)$$

  During validation, the tester deliberately induces consumption anomalies by withholding or reducing the feed placed in the trough, simulating a sick pig's reduced intake. Each induced event is logged manually alongside the system's automated response. A *True Positive* is counted when the Analytical Engine correctly flags the induced deviation within one polling cycle (≤ 1 second). Any induced event that does not trigger an alert is classified as a *False Negative*. The ARA is then computed as the ratio of correctly detected events to total induced events across all test runs.
- **System Uptime (U_{sys}):** Measured over a 72-hour continuous operation trial, consistent with endurance validation practices for embedded monitoring systems [7], [9]:

$$U_{sys} = \frac{T_{total} - T_{downtime}}{T_{total}} \times 100\% \quad (10)$$

  where T_{total} is the total trial duration in hours and $T_{downtime}$ is the cumulative time during which sensor polling or the dashboard was unavailable. In the system, a Python watchdog thread periodically writes a heartbeat timestamp to the SQLite database. $T_{downtime}$ is computed post-trial by identifying gaps in the heartbeat log that exceed a predefined timeout interval (e.g., > 10 seconds), and summing their durations.
- **Data Display Latency:** Time elapsed between a sensor reading being acquired at the feeder node and its appearance on the dashboard, measured across 50 consecutive poll cycles. Target threshold: ≤ 2 seconds [7], [9].
- **SMS Delivery Rate (SDR):** The end-to-end percentage of alert messages successfully delivered as SMS to the caretaker's mobile phone via the LoRa-GSM gateway, compared to the total number of alerts generated by the central unit. A minimum passing threshold of $\geq 95\%$ is required to confirm reliable remote notification delivery [7], [14]:

$$SDR = \frac{\text{Total SMS Delivered}}{\text{Total Alerts Generated}} \times 100\% \quad (11)$$

  During testing, the system logs each alert event with a unique identifier in the SQLite database at the moment of LoRa transmission. The caretaker's mobile phone is used to count the total number of SMS messages received, each of which contains the corresponding alert identifier. The SDR is then computed by dividing the number of confirmed SMS receipts by the total number of LoRa-transmitted alerts logged in the database.
- **End-to-End Alert Latency:** The time elapsed between an alert being generated by the Analytical Engine and the corresponding SMS being successfully delivered to the caretaker's mobile phone, encompassing LoRa transmission, gateway processing, and GSM delivery. Target threshold: ≤ 30 seconds per alert message [7], [14].

K. Testing and Checking

The testing protocol is designed to systematically verify that each of the project's four specific objectives has been accomplished. Each test is explicitly mapped to its corresponding objective, and the pass/fail criteria are defined such that successful completion of all tests constitutes a complete demonstration of the system's intended functionality. This structured approach ensures that panelists and evaluators can trace every test result directly to the objective it validates [7]–[9].

Controlled Testing Environment. All tests described in this section are designed to be conducted entirely in a controlled laboratory or bench-testing environment, without requiring deployment to an actual piggery. This allows evaluators and panelists to assess the full system performance during a demonstration session. The real-world conditions of a piggery are simulated as follows:

- **Feed consumption simulation:** Instead of live pigs consuming feed, the tester manually removes known quantities of feed from the trough between feeding sessions using a calibrated reference scale. This simulates normal consumption patterns. Anomalous consumption (e.g., a sick pig not eating) is simulated by deliberately withholding or reducing the amount of feed removed, causing the rolling-window baseline to detect a deviation.
- **Water level simulation:** The water trough is filled to a known level, and water is manually drained using a valve or siphon to simulate pig drinking. The system's solenoid refill response is then observed and verified. Anomalous water consumption is simulated by draining the trough below the configured threshold more rapidly than normal.
- **Air quality simulation:** Ammonia threshold exceedances are simulated by exposing the MQ-135 sensor to a controlled ammonia test gas source (e.g., a diluted ammonia cleaning solution in an enclosed chamber), or by using a known volatile compound that triggers the sensor above its configured threshold, then removing the source to verify the alert clears.
- **LoRa-SMS notification simulation:** The LoRa-GSM gateway is positioned at a measured distance from the main control module within the test venue. Anomalies are induced as described above, and the evaluator verifies SMS receipt on the caretaker's registered mobile phone in real time.
- **Continuous operation simulation:** For the 72-hour uptime and data persistence tests, the system is run on a benchtop with periodic automated or manual sensor interactions to generate representative data volumes, verifying that the database and dashboard remain operational over extended periods.

This controlled approach ensures that every aspect of the system—dispensing accuracy, sensor reliability, anomaly detection, dashboard functionality, and remote SMS notification—can be fully validated by evaluators in a single demonstration session without logistical dependence on a live piggery environment.

Objective 1: Implement a programmable central unit to manage main storage and automate the dispensing process.

This objective requires demonstrating that the central unit (Raspberry Pi and hopper assembly) can store feed, execute a programmable feeding schedule, and dispense accurate quantities of feed to each feeder module without manual intervention.

- **Test 1.1 – Feed Dispensing Accuracy (FDA):** The auger-based dispensing mechanism will be tested across 30 dispensing events per feeder module. For each event, the target mass M_{target} is set via the dashboard's feeding schedule, and the actual dispensed mass M_{actual} is measured by the load cell. The FDA for each event is computed as:

$$FDA = \left(1 - \frac{|M_{target} - M_{actual}|}{M_{target}}\right) \times 100\% \quad (12)$$

  Pass Criterion: Mean FDA across all 30 trials $\geq 95\%$.
  How this demonstrates the objective: If the system consistently dispenses feed within $\pm 5\%$ of the programmed target across repeated trials, it confirms that the central unit can reliably automate the dispensing process with the accuracy required for precision feeding. The use of 30 trials ensures statistical robustness and accounts for natural variability in granular feed flow [8], [9].
- **Test 1.2 – Schedule Execution Verification:** A 24-hour automated feeding schedule consisting of three feeding events at pre-configured times will be programmed into the dashboard. The system must autonomously initiate each dispensing event within ± 30 seconds of the scheduled time without manual intervention.
  Pass Criterion: All three scheduled feeding events are executed within the time tolerance window, and the corresponding dispensed masses are logged in the SQLite database with correct timestamps.
  How this demonstrates the objective: Successful execution of all scheduled events proves that the central unit is fully programmable and can manage the dispensing process autonomously over a sustained period, fulfilling the core requirement of automated feed management.

Objective 2: Design individual feeder modules capable of monitoring feed consumption, water availability, and ammonia levels.

This objective requires demonstrating that each feeder module's sensors produce accurate, reliable readings for feed weight, water level, and air quality under realistic conditions.

- **Test 2.1 – Load Cell Calibration and Accuracy:** Each load cell and HX711 assembly will be calibrated using standard reference weights across a range of 0 to 5 kg. A minimum of 10 calibration trials per load cell will be conducted at five weight increments (1 kg, 2 kg, 3 kg, 4 kg, 5 kg). Linearity (R^2) and hysteresis will be evaluated.
  Pass Criterion: Linear regression $R^2 \geq 0.99$ across the tested range, and all individual readings within $\pm 2\%$ of the reference weight.

How this demonstrates the objective: Accurate load cell readings are the foundation for feed consumption monitoring. A high R^2 confirms that the sensor produces linear, repeatable measurements, ensuring that the feeder module can reliably track how much feed a pig consumes per session [8], [9].

- **Test 2.2 – Water Level Sensor Validation:** The ultrasonic level sensors will be validated against manual depth measurements using a calibrated dipstick across a range of operational trough depths (5 cm to 25 cm). A minimum of 10 trials per sensor will be conducted.
  Pass Criterion: All sensor readings within ± 1 cm of the manual dipstick measurement, and the solenoid valve correctly triggers when the water level falls below the programmed threshold.
  How this demonstrates the objective: Accurate water level readings confirm that the feeder module can reliably monitor water availability. Correct solenoid triggering proves the module can maintain adequate water supply without manual intervention [7], [9].
- **Test 2.3 – Air Quality Sensor Calibration (MQ-135):** The MQ-135 sensors will undergo a minimum 48-hour burn-in period followed by calibration in a controlled, clean-air environment to establish a baseline resistance (R_0). Sensors will then be tested against known ammonia concentrations in the range of 10 to 300 ppm to verify threshold response accuracy.
  Pass Criterion: The sensor correctly triggers an alert when ammonia concentration exceeds the configured threshold (default: 25 ppm) in at least 9 out of 10 controlled exposure trials.
  How this demonstrates the objective: Reliable threshold detection confirms that the feeder module can monitor ammonia levels and identify hazardous air quality conditions, fulfilling the environmental monitoring requirement of this objective [9], [10], [12].

Objective 3: Develop a user interface on the central unit for system monitoring, data acquisition, and historical logging.

This objective requires demonstrating that the touchscreen dashboard displays real-time sensor data, allows configuration of feeding schedules, and maintains a persistent historical log accessible to the caretaker.

- **Test 3.1 – Data Display Latency:** The time elapsed between a sensor reading being acquired at the feeder node and its appearance on the touchscreen dashboard will be measured across 50 consecutive poll cycles.
  Pass Criterion: Mean latency $\leq$ 2 seconds, with no individual reading exceeding 5 seconds.
  How this demonstrates the objective: A latency of ≤ 2 seconds confirms that the dashboard provides real-time monitoring, ensuring the caretaker sees current data rather than stale readings. This validates the system monitoring component of the objective [7], [9].
- **Test 3.2 – Data Persistence and Historical Logging:** The system will be operated continuously for 72 hours. After the trial, the SQLite database will be queried to verify that all sensor readings (feed weight, water level, ammonia), dispensing events, and anomaly alerts have been logged with correct timestamps.
  Pass Criterion: Zero missing entries in the database relative to the total number of expected polling cycles ($\geq 99\%$ data completeness), and all entries retrievable through the dashboard's historical view.
  How this demonstrates the objective: Complete and accurate data persistence over a 72-hour window confirms that the system fulfills the data acquisition and historical logging requirements. The ability to retrieve and display historical data on the dashboard validates the full data pipeline from sensor to screen [7], [9].
- **Test 3.3 – System Uptime (U_{sys}):** Measured over the same 72-hour continuous operation trial to verify sustained availability:

$$U_{sys} = \frac{T_{total} - T_{downtime}}{T_{total}} \times 100\% \tag{13}$$

  where T_{total} is the total trial duration and $T_{downtime}$ is the cumulative time during which sensor polling or the dashboard was unavailable.
  Pass Criterion: $U_{sys} \geq 95\%$.
  How this demonstrates the objective: A sustained uptime of $\geq 95\%$ confirms that the user interface and data acquisition pipeline are robust enough for continuous farm operation, where downtime directly translates to missed monitoring windows and potential undetected anomalies.

Objective 4: Integrate an SMS module to alert caretakers when the system detects consumption or environmental anomalies.

This objective requires demonstrating that the LoRa-to-SMS notification pipeline reliably detects anomalies and delivers timely SMS alerts to the caretaker's mobile phone.

- **Test 4.1 – Alert Response Accuracy (ARA):** Consumption anomalies will be deliberately induced by withholding or reducing the feed placed in the trough, simulating a sick pig's reduced intake. A total of 20 anomaly events will be induced across all feeder modules.

$$ARA = \frac{\text{True Positive Alerts}}{\text{Total Induced Events}} \times 100\% \tag{14}$$

  Pass Criterion: $ARA \geq 90\%$.
  How this demonstrates the objective: A high ARA confirms that the Analytical Engine's rolling-window algorithm reliably detects consumption and environmental anomalies—the prerequisite for any alert to be generated. Without accurate detection, the SMS module cannot fulfill its purpose [10], [11].
- **Test 4.2 – SMS Delivery Rate (SDR):** For each detected anomaly, the system will transmit a LoRa alert to the remote LoRa-GSM gateway, which forwards it as an

SMS. Each SMS contains a unique alert identifier for cross-referencing.

$$SDR = \frac{\text{Total SMS Delivered}}{\text{Total Alerts Generated}} \times 100\% \tag{15}$$

Pass Criterion: $SDR \geq 95\%$.

How this demonstrates the objective: An SDR of $\geq 95\%$ confirms that the end-to-end LoRa-to-SMS pipeline reliably delivers alerts to the caretaker's mobile phone. This directly validates the integration of the SMS module as specified in the objective [7], [14].

- **Test 4.3 – End-to-End Alert Latency:** The time elapsed between the Analytical Engine generating an alert and the SMS being received on the caretaker's phone will be measured for each alert event. This encompasses LoRa transmission, gateway processing, and GSM delivery.
  Pass Criterion: Mean latency ≤ 30 seconds per alert.
  How this demonstrates the objective: A delivery latency of ≤ 30 seconds confirms that the SMS alerts are timely enough for the caretaker to take immediate corrective action, validating that the system provides meaningful, real-time remote notification capability [7], [14].
- **Test 4.4 – LoRa Communication Range:** The LoRa link between the main control module and the LoRa-GSM gateway will be tested at increasing distances (e.g., 50 m, 100 m, 200 m, 500 m) within the farm environment. At each distance, 20 alert payloads will be transmitted and the Received Signal Strength Indicator (RSSI) and delivery success rate will be recorded.
  Pass Criterion: $RSSI \geq -120$ dBm and delivery success rate $\geq 95\%$ at the maximum operational distance required by the farm layout.
  How this demonstrates the objective: Confirming adequate LoRa range ensures that the SMS module can function across the actual farm property, validating that the alert system works not just in laboratory conditions but in the intended deployment environment [7], [14].

Communication Infrastructure Validation:

- **Test 5.1 – RS-485 Signal Integrity and MODBUS Reliability:** The RS-485 wired bus will undergo signal integrity testing with all feeder nodes active simultaneously. A digital storage oscilloscope will verify that differential voltage levels meet the RS-485 standard minimum of ± 1.5 V. MODBUS RTU packet loss rates will be quantified under simulated electrical noise conditions at the maximum prototype cable length.
  Pass Criterion: Packet loss rate $\leq 1\%$ across all feeder nodes under full bus load.
  How this demonstrates system readiness: While not mapped to a single specific objective, reliable wired communication underpins all four objectives. If the RS-485 bus fails, sensor data cannot reach the central unit, dispensing commands cannot be issued, the dashboard cannot display data, and alerts cannot be generated. This test confirms the foundational infrastructure upon which all objectives depend [7], [9].

L. Deployment Plan

The transition from the laboratory prototype to field operation in a commercial piggery will follow a structured, four-phase deployment strategy to minimize disruption to existing farm operations:

- **Phase 1: Site Assessment and Electrical Routing:** An evaluation of the piggery's physical layout will determine the optimal routing for the RS-485 twisted-pair cables and DC power distribution. Cable trajectories will be explicitly planned to avoid electromagnetic interference (EMI) from existing high-voltage farm equipment or large motors. This phase must produce three deliverables before installation proceeds: a cable routing diagram, a DC power distribution plan, and an EMI risk assessment identifying high-interference zones within the facility.
- **Phase 2: Physical Installation:** The central 0.18 m^3 hopper, auger mechanism, and main control module (Raspberry Pi) will be securely mounted at the centralized feeding station. Individual feeder nodes (ESP32 enclosures, load cells, and level sensors) will be mechanically integrated into the pen structures. The level sensors will be securely mounted directly above the water troughs, and the plumbing will be retrofitted to integrate the automated solenoid valves. Installation is considered complete only after all nodes pass a hardware checklist confirming that all units are powered, all sensors are returning non-zero readings, and all MODBUS device addresses have been successfully assigned and acknowledged by the master.
- **Phase 3: Commissioning and Baseline Initialization:** Once powered, unique MODBUS device addresses will be assigned to each feeder node. The system will initially operate in a "silent monitoring" phase (e.g., 7 to 14 days). During this period, the system will dispense feed and log data to the SQLite database without triggering health alerts, allowing the rolling-window algorithm to establish accurate, pen-specific consumption baselines (μ_N). Anomaly detection will not be activated until a minimum of $N = 14$ feeding sessions have been recorded per module, ensuring the rolling-window baseline is statistically sufficient before alerts are generated.
- **Phase 4: Caretaker Turnover and Training:** Farm operators will undergo practical training on utilizing the Raspberry Pi touchscreen interface. This includes instructing them on how to interpret the time-series charts, modify feeding schedules, address alert logs, and perform routine maintenance such as clearing auger jams or cleaning the optical/gas sensor interfaces. Turnover is considered complete only when each caretaker can independently demonstrate three competencies without assistance: interpreting and responding to a generated alert, modifying a feeding schedule on the dashboard, and performing basic sensor maintenance.

REFERENCES

[1] Piyush, "Animal husbandry statistics 2025: Global livestock numbers, meat and dairy production trends," November 2025.

[2] "Global, asian, and philippine livestock production systems: A comprehensive sectoral analysis of swine, poultry, and bovine dynamics (2024–2026)," 2026.
[3] OECD and Food and Agriculture Organization of the United Nations (FAO), "Oecd-fao agricultural outlook 2025-2034," tech. rep., OECD Publishing, July 2025.
[4] USDA Foreign Agricultural Service, "Production - pork," 2026.
[5] porciNews, "Philippine pig production rises late in 2025, but yearly output falls," February 2026.
[6] R. H. Lo, "Rebooting the philippines swine industry: Challenges, prospects, and policy directions," September 2025.
[7] Beyond Technology, "Benefits of smart monitoring in pig farming beyond visual control," June 2025.
[8] J. H. B. Dingcong, J. H. Tangoan, W. S. O. Tiu, J. E. L. Undag, R. K. V. Villegas, C. M. I. Gohetia, and J. D. A. Campanera, "Monitoring the feeder and drinker visits of pigs using image processing," in *Department of Computer Engineering*, (Cebu City, Philippines), University of San Carlos, 2017.
[9] M. O. Adeoye, O. T. Arogundade, and T. Olaleye, "Animalia: An iot-based temperature monitoring system for piggery farming," *IRE Journals*, vol. 9, September 2025.
[10] E. Sadeghi, C. Kappers, A. Chiumento, M. Derks, and P. Havinga, "Improving piglets health and well-being: A review of piglets health indicators and related sensing technologies," *Smart Agricultural Technology*, vol. 5, p. 100246, 2023.
[11] P. Racewicz, A. Ludwiczak, E. Skrzypczak, J. Składanowska-Baryza, H. Biesiada, T. Nowak, S. Nowaczewski, M. Zaborowicz, M. Stanisz, and P. Ślósarz, "Welfare health and productivity in commercial pig herds," *Animals*, vol. 11, April 2021.
[12] P. Yamsakul, T. Yano, K. Junchum, W. Anukul, and N. Kittiwan, "Machine learning-based detection of pig coughs and their association with respiratory diseases in fattening pigs," *Preprints.org*, June 2025.
[13] HealthforAnimals and Food and Agriculture Organization of the United Nations (FAO), "How prevention can reduce the need for antibiotics: Pathways to reduce the need for antimicrobials on farms for sustainable agrifood systems transformation (renofarm)," tech. rep., HealthforAnimals and FAO, Brussels and Rome, 2024.
[14] M. N. Reza, M. R. Ali, M. A. Haque, H. Jin, H. Kyoung, Y. K. Choi, G. Kim, and S.-O. Chung, "A review of sound-based pig monitoring for enhanced precision production," *Journal of Animal Science and Technology*, vol. 67, no. 2, pp. 277–302, 2025.